

Plano Técnico — Ferramenta Web de Validação de Orçamentos

1. Visão geral e objetivos

Objetivo do projeto

Construir uma ferramenta web interna, em **Next.js + React + Tailwind CSS + Supabase**, para que a equipe humana valide, corrija, rejeite ou aprove orçamentos criados automaticamente por um chatbot de WhatsApp.

A ferramenta terá dois ambientes principais:

1. **Kanban de acompanhamento**
Visão operacional dos atendimentos por status.
2. **Card de validação**
Tela detalhada para revisar dados do paciente, exames, termos não encontrados e aprovar ou rejeitar o orçamento.

Premissas

O backend já existe e deve ser preservado:

- Banco Supabase/PostgreSQL já modelado.
- RLS já habilitado.
- Bot já grava atendimentos.
- Trigger já recalcula atendimentos.total_validado.
- Webhook já dispara envio ao cliente quando status = 'validado'.
- Frontend usará @supabase/supabase-js.
- Autenticação será feita por **login único compartilhado** da equipe.

Fora de escopo

Não implementar agora:

- Novo fluxo de agendamento.
- Novos status.
- Alterações no bot/n8n.
- Alterações no webhook.
- Enriquecimento do catálogo de exames.
- Regras novas de preço.
- Cálculo de total no front.
- Controle individual real de usuários no Supabase Auth.

2. Arquitetura

Diagrama textual

Usuário interno



Next.js App



React Components + Tailwind UI



supabase-js client



Supabase Auth — login compartilhado



Postgres via RLS

└─ atendimentos

└─ atendimento_exames

└─ catalogo_exames



Triggers existentes

└─ recalculam atendimentos.total_validado



Supabase Realtime

└─ atualiza Kanban e status em tempo real



Database Webhook existente

└─ ao status = validado, envia orçamento no WhatsApp

Decisão principal de arquitetura

Opção A — Front acessa Supabase diretamente com supabase-js

Vantagens:

- Mais simples.
- Menor latência.
- Menos código backend.
- Usa RLS já existente.

- Adequado ao escopo atual.

Desvantagens:

- A lógica de validação fica mais exposta no client.
- Não há camada intermediária para auditoria refinada.
- Com login compartilhado, o banco não sabe qual pessoa real fez a ação.

Opção B — Criar API Routes/Server Actions no Next.js

Vantagens:

- Centraliza regras de negócio.
- Melhor para auditoria futura.
- Permite usar validações server-side mais rígidas.

Desvantagens:

- Mais complexidade.
- Pode duplicar regras já protegidas por RLS.
- Exige mais manutenção.

Recomendação

Para o MVP, usar **Opção A: acesso direto via supabase-js no client**, com RLS existente e sem expor service_role.

Usar API Routes apenas futuramente se houver necessidade de:

- auditoria forte por usuário real;
- validações centralizadas;
- integração com outros serviços;
- logs detalhados de ações críticas.

3. Camada de dados

3.1 Entidades usadas

atendimentos

Usada como card principal.

Colunas relevantes:

- id
- protocolo
- telefone

- responsavel_nome
- paciente_nome
- paciente_cpf
- paciente_nascimento
- paciente_cidade
- plano_convenio
- unidade_preferida
- agendamento_desejado
- medico_solicitante
- qualidade_pedido_imagem
- observacoes_prescricao
- observacoes_conversa
- total_bruto
- total_validado
- status
- termos_nao_encontrados
- cotacao_original
- validado_por
- validado_em
- enviado_em
- created_at
- updated_at

atendimento_exames

Usada para a lista editável de exames.

Colunas relevantes:

- id
- atendimento_id
- sku
- nome
- preco
- prazo_dias

- jejum_horas
- preparo
- termo_original
- match_por
- score
- ambiguo
- incluido
- editado_manual
- ordem

catalogo_exames

Usada somente para autocomplete.

Colunas relevantes:

- sku
- nome
- sinonimos
- preco
- material
- metodo
- preparo
- prazo_dias
- jejum_horas

4. Contratos de leitura e escrita

4.1 Kanban — leitura inicial

Objetivo

Carregar cards resumidos por status.

Tabela

atendimentos

Filtro

Status dentro de:

- aguardando_validacao

- em_validacao
- validado
- enviado
- rejeitado

Colunas lidas

- id
- protocolo
- paciente_nome
- telefone
- responsavel_nome
- medico_solicitante
- total_validado
- total_bruto
- status
- created_at
- updated_at

Ordenação recomendada

- created_at DESC dentro de cada coluna.

Observação

O Kanban deve usar total_validado quando existir. Se estiver nulo, exibir total_bruto como referência visual, com indicação de que ainda não foi validado.

4.2 Card de validação — leitura do atendimento

Objetivo

Carregar os dados completos de um atendimento.

Tabela

atendimentos

Filtro

- id = atendimento_id

Colunas lidas

Todas as colunas listadas em atendimentos.

Comportamento

Ao abrir o card, antes de permitir edição efetiva, a web deve tentar mudar:

status: aguardando_validacao → em_validacao

Essa transição deve ser feita de forma condicional:

- atualizar apenas se o status atual for aguardando_validacao;
 - se o status já for em_validacao, mostrar aviso de possível conflito;
 - se o status já for validado, enviado ou rejeitado, abrir em modo restrito ou leitura.
-

4.3 Card de validação — leitura dos exames

Tabela

atendimento_exames

Filtro

- atendimento_id = atendimento.id

Colunas lidas

- id
- atendimento_id
- sku
- nome
- preco
- prazo_dias
- jejum_horas
- preparo
- termo_original
- match_por
- score
- ambiguo
- incluido
- editado_manual
- ordem

Ordenação

- ordem ASC

- fallback: created_at ASC, caso exista no banco
 - se não houver created_at, usar nome ASC como fallback visual
-

4.4 Card de validação — atualizar dados do paciente

Tabela

atendimentos

Campos editáveis recomendados

- paciente_nome
- paciente_cpf
- paciente_nascimento
- paciente_cidade
- plano_convenio
- unidade_preferida
- agendamento_desejado
- observacoes_conversa

Campos opcionalmente editáveis

- medico_solicitante
- observacoes_prescricao

Recomendação

Manter o bloco de prescrição preferencialmente **somente leitura**, com exceção de medico_solicitante e observacoes_prescricao se a operação realmente precisar corrigir esses campos.

Trade-off:

- **Somente leitura:** preserva fidelidade ao que o bot extraiu.
- **Editável:** melhora operação humana, mas pode distorcer auditoria.

Recomendação prática:

- deixar medico_solicitante editável;
 - deixar qualidade_pedido_imagem somente leitura;
 - deixar observacoes_prescricao editável com indicação de “observação validada pela equipe”.
-

4.5 Card de validação — atualizar exame

Tabela

atendimento_exames

Campos editáveis

- preco
- prazo_dias
- jejum_horas
- preparo
- incluido

Campos alterados automaticamente pela web

Sempre que houver alteração manual relevante em um exame:

- editado_manual = true

Campos somente leitura

- sku
- nome, salvo se a equipe precisar corrigir manualmente
- termo_original
- match_por
- score
- ambiguo

Recomendação sobre nome

Permitir edição manual de nome somente em modo “edição avançada”.

Motivo:

- o ideal é adicionar exame correto via catalogo_exames;
- editar nome livremente pode criar divergência entre orçamento e catálogo.

4.6 Card de validação — adicionar exame pelo catálogo

Leitura do catálogo

Tabela:

catalogo_exames

Busca por:

- nome
- sku

- sinonimos

Colunas carregadas:

- sku
- nome
- sinonimos
- preco
- material
- metodo
- preparo
- prazo_dias
- jejum_horas

Inserção em atendimento_exames

Ao escolher um item do catálogo, inserir nova linha com:

- atendimento_id = atendimento.id
- sku = catalogo_exames.sku
- nome = catalogo_exames.nome
- preco = catalogo_exames.preco
- preparo = catalogo_exames.preparo
- prazo_dias = catalogo_exames.prazo_dias
- jejum_horas = catalogo_exames.jejum_horas
- incluido = true
- editado_manual = true
- ordem = próxima ordem disponível

Trade-off da busca

Busca simples no Supabase

- Usar filtro textual por nome, sku e sinonimos.
- Mais simples.
- Suficiente para MVP.

Busca com índice full-text ou RPC

- Melhor performance e relevância.
- Exige mudança/ajuste no banco.

Recomendação

MVP com busca simples. Se o catálogo for grande ou a busca ficar ruim, solicitar melhoria posterior no banco com índice full-text ou função RPC.

4.7 Card de validação — remover exame

Há duas opções:

Opção A — excluir linha de atendimento_exames

Vantagem:

- Remove de fato o exame errado.
- Interface fica limpa.

Desvantagem:

- Pode perder rastreabilidade do que o bot sugeriu.

Opção B — marcar incluído = false

Vantagem:

- Mantém histórico do exame sugerido.
- Preserva rastreio para auditoria.
- O trigger já ignora exames com incluído = false.

Desvantagem:

- A lista pode ficar mais poluída.

Recomendação

Para o botão principal, usar **“Remover do orçamento” = incluído = false**.

Se houver necessidade operacional, criar uma ação secundária “Excluir definitivamente”, restrita e com confirmação. Como a regra diz “remover”, a implementação pode permitir delete, mas a recomendação técnica é preservar histórico via incluído = false.

4.8 Validar atendimento

Tabela

atendimentos

Atualização

- status = 'validado'
- validado_por = nome informado no seletor/campo de validador
- validado_em = timestamp atual

Observação crítica

Essa ação dispara o webhook já existente de envio ao WhatsApp.

Logo, a UI deve exigir:

- confirmação explícita;
- estado de carregamento;
- bloqueio contra duplo clique;
- mensagem clara de consequência externa.

Texto sugerido:

Ao validar, o orçamento será enviado ao cliente pelo WhatsApp. Deseja continuar?

4.9 Rejeitar atendimento

Tabela

atendimentos

Atualização

- status = 'rejeitado'

Recomendação

O modelo atual não possui campo de motivo de rejeição.

Para o MVP:

- permitir rejeição simples;
- exibir confirmação;
- se precisar registrar motivo, listar como dependência de banco.

Dependência opcional:

- adicionar coluna motivo_rejeicao text;
 - ou criar tabela de histórico/auditoria.
-

5. Máquina de estados do status

Estados existentes

aguardando_validacao

em_validacao

validado

enviado

rejeitado

Transições acionadas pela web

aguardando_validacao → em_validacao

em_validacao → validado

em_validacao → rejeitado

Transições externas à web

validado → enviado

Essa transição deve ser entendida como provável efeito do backend/webhook, não da ferramenta web.

Regras por transição

5.1 Abrir card

Transição:

aguardando_validacao → em_validacao

Efeito:

- protege o atendimento contra novas escritas do bot;
- indica que alguém iniciou validação;
- atualiza Kanban em tempo real.

Regra:

- fazer update condicional somente se status atual for aguardando_validacao.

Se falhar:

- buscar status atual;
- exibir mensagem: “Este atendimento já está em validação ou já foi finalizado.”

5.2 Validar

Transição:

em_validacao → validado

Efeito:

- dispara webhook;
- cliente recebe orçamento validado;
- card sai da coluna em_validacao para validado;
- posteriormente pode ir para enviado.

Regras:

- exigir confirmação;
 - impedir se não houver telefone;
 - recomendar validador preenchido;
 - exibir total validado lido do banco;
 - não calcular total no front.
-

5.3 Rejeitar

Transição:

em_validacao → rejeitado

Efeito:

- atendimento não será enviado ao cliente;
- card sai do fluxo de validação.

Regras:

- exigir confirmação;
 - idealmente registrar motivo, mas o banco atual não possui campo.
-

5.4 Reabrir card validado

Não recomendado no MVP.

Motivo:

- validado já pode ter disparado envio ao WhatsApp;
- editar depois disso pode criar divergência entre o que foi enviado e o que aparece na web.

Comportamento recomendado:

- abrir como somente leitura;
- mostrar aviso: “Atendimento já validado. Alterações não são permitidas nesta versão.”

Dependência futura:

- criar status ou rotina de reprocessamento/reenvio, se necessário.
-

6. Detalhamento por tela

6.1 Aba 2 — Kanban de acompanhamento

Embora seja chamada de Aba 2, ela deve ser a tela inicial operacional.

Objetivo

Permitir que a equipe veja rapidamente:

- novos atendimentos;
- atendimentos em validação;
- orçamentos validados;
- orçamentos enviados;
- rejeições.

Wireframe textual

[Topo]

Logo / Nome do sistema

Usuário logado

Campo "Validador atual"

Botão atualizar manualmente

[Filtros]

Busca por protocolo, paciente, telefone ou médico

Filtro por data

Filtro por unidade

Filtro por convênio

[Kanban]

| Aguardando | Em validação | Validado | Enviado | Rejeitado |
| contador | contador | contador | contador | contador |

| Card | Card | Card | Card | Card |
| Card | Card | Card | Card | Card |

Card resumido

Cada card deve exibir:

- protocolo
- paciente_nome
- telefone
- total_validado ou total_bruto
- created_at
- medico_solicitante
- indicador de termos_nao_encontrados
- indicador de exames ambíguos, se carregado por agregação futura

Componentes

- KanbanPage
- KanbanBoard
- KanbanColumn
- AtendimentoCard
- KanbanFilters
- StatusBadge
- RealtimeIndicator
- EmptyColumnState

Estados de UI

Loading

- skeleton cards;
- colunas já renderizadas;
- indicador “carregando atendimentos”.

Erro

- mensagem objetiva;
- botão “tentar novamente”;
- log técnico no console apenas em desenvolvimento.

Vazio

Por coluna:

- “Nenhum atendimento aguardando validação.”
- “Nenhum atendimento em validação.”
- etc.

Comportamento ao clicar no card

Ao clicar:

1. navegar para /atendimentos/[id];
2. tela do card tenta transição para em_validacao;
3. se conseguir, liberar edição;
4. se não conseguir, abrir conforme status atual.

6.2 Aba 1 — Card de validação

Objetivo

Permitir revisão completa de um atendimento antes do envio ao cliente.

Wireframe textual

[Topo fixo]

Protocolo

Status

Data de criação

Telefone

Responsável

Total validado

Botões: Rejeitar | Validar

[Alerta]

Termos não encontrados

Exames ambíguos

Possíveis inconsistências

[Bloco Paciente]

Nome

CPF

Nascimento

Cidade

Convênio

Unidade preferida

Agendamento desejado

[Bloco Prescrição]

Médico solicitante

Qualidade da imagem

Observações da prescrição

[Observações da conversa]

Texto livre coletado do cliente

[Exames]

Busca no catálogo para adicionar exame

Tabela/lista:

Incluído?

Nome

SKU

Preço

Prazo

Jejum

Preparo

Termo original

Match por

Score

Ambíguo

Ações

[Auditoria]

Cotação original

Campos de rastreo

Componentes

- AtendimentoValidationPage
 - AtendimentoHeader
 - PatientForm
 - PrescriptionPanel
 - ConversationNotes
 - TermsNotFoundAlert
 - ExamList
 - ExamRow
 - ExamEditorDrawer ou ExamInlineEditor
 - CatalogAutocomplete
 - TotalSummary
 - OriginalQuoteViewer
 - ValidationActions
 - ConfirmValidateDialog
 - ConfirmRejectDialog
-

6.3 Bloco do paciente

Campos editáveis

- paciente_nome
- paciente_cpf
- paciente_nascimento
- paciente_cidade
- plano_convenio
- unidade_preferida
- agendamento_desejado

Validações

- Nome: não obrigatório no banco, mas mostrar alerta se vazio.
- CPF: permitir texto, pois a coluna é text; validar formato apenas visualmente.
- Nascimento: manter texto; não converter automaticamente para data.
- Cidade: texto livre.

- Convênio: texto livre.
- Unidade: texto livre ou lista fixa se o negócio já tiver unidades padronizadas.
- Agendamento desejado: texto livre.

Trade-off sobre validação rígida

Não converter paciente_nascimento para date, porque o banco usa text e exemplos como "15/07/70" podem ter ambiguidade.

Recomendação:

- aplicar máscaras leves;
 - não bloquear salvamento por formatação imperfeita;
 - exibir alertas de qualidade.
-

6.4 Bloco da prescrição

Campos

- medico_solicitante
- qualidade_pedido_imagem
- observacoes_prescricao

Recomendação

- qualidade_pedido_imagem: somente leitura.
- medico_solicitante: editável.
- observacoes_prescricao: editável com cuidado.

Motivo

A equipe pode corrigir nome de médico e observações, mas a qualidade da imagem é um dado de rastreio da extração do bot.

6.5 Observações da conversa

Campo:

- observacoes_conversa

Recomendação:

- editável;
 - salvar com debounce ou botão “Salvar alterações”;
 - mostrar que essa observação não altera automaticamente exames.
-

6.6 Lista de exames

Exibição

Cada exame deve mostrar:

- checkbox incluído
- nome
- sku
- preco
- prazo_dias
- jejum_horas
- preparo
- termo_original
- match_por
- score
- ambiguo
- editado_manual

Destaques visuais

- ambiguo = true: badge amarelo ou alerta.
- score baixo: mostrar discretamente.
- editado_manual = true: badge “editado”.
- incluído = false: card riscado ou colapsado em seção “fora do orçamento”.

Edição

Campos editáveis:

- preco
- prazo_dias
- jejum_horas
- preparo
- incluído

Ao editar:

- atualizar linha em atendimento_exames;
- definir editado_manual = true;
- aguardar trigger recalculando total_validado;

- recarregar atendimento ou receber update por Realtime.

Remoção

Ação principal:

- “Remover do orçamento”
- grava incluído = false.

Ação opcional:

- “Excluir definitivamente”
 - exige confirmação dupla.
 - não recomendada como padrão.
-

6.7 Termos não encontrados

Campo:

- termos_ nao_encontrados

Tipo:

- jsonb, esperado como array.

Exibição

Mostrar em destaque no topo da validação:

Termos não encontrados pelo bot:

- termo 1
- termo 2
- termo 3

Para cada termo:

- botão “buscar no catálogo”;
- ao clicar, preencher autocomplete com o termo;
- equipe seleciona exame correto;
- inserir em atendimento_examens.

Regra

Não alterar diretamente termos_ nao_encontrados no MVP.

Motivo:

- ele é evidência do comportamento do bot;
- serve para auditoria e melhoria futura do catálogo.

6.8 Total

Campo:

- total_validado

Regra crítica

A web **nunca calcula nem grava** total_validado.

Comportamento correto:

1. equipe edita exames;
2. banco recalcula via trigger;
3. web relê atendimentos.total_validado;
4. UI exibe o valor atualizado.

UI

Mostrar:

- total_bruto, como referência do bot;
- total_validado, como valor final;
- diferença entre bruto e validado, apenas calculada visualmente se necessário.

A diferença visual pode ser calculada no front, mas não deve ser persistida.

6.9 Ações finais

Validar

Exigir modal:

Você está prestes a validar este orçamento.

Essa ação enviará a resposta ao cliente pelo WhatsApp.

Total validado: R\$ X

Telefone: Y

Deseja continuar?

Campos no modal:

- seletor/campo validado_por
- confirmação final

Bloqueios recomendados:

- bloquear se telefone vazio;

- alertar se não houver exames incluídos;
- alertar se total_validado for zero;
- alertar se houver termos_ao_encontrados não tratados;
- permitir override apenas com confirmação explícita.

Rejeitar

Exigir modal:

Este atendimento será rejeitado e não seguirá para envio.

Deseja continuar?

Como não há campo de motivo, não gravar motivo no MVP.

7. Organização de pastas do Next.js

Assumindo App Router.

src/

app/

layout.tsx

page.tsx

login/

page.tsx

kanban/

page.tsx

atendimentos/

[id]/

page.tsx

components/

layout/

AppShell

Header

Sidebar

UserValidatorSelector

kanban/

KanbanBoard

KanbanColumn

AtendimentoCard

KanbanFilters

StatusBadge

RealtimeIndicator

atendimento/

AtendimentoHeader

PatientForm

PrescriptionPanel

ConversationNotes

TermsNotFoundAlert

TotalSummary

OriginalQuoteViewer

exames/

ExamList

ExamRow

ExamEditor

CatalogAutocomplete

IncludedToggle

ExamTraceInfo

dialogs/

ConfirmValidateDialog

ConfirmRejectDialog

ConflictDialog

ui/

Button

Input

Textarea

Select

Badge

Card

Modal

Skeleton

Toast

lib/

supabase/

client

types

formatters/

currency

date

cpf

validators/

atendimento

exame

constants/

status

routes

hooks/

useAtendimentosKanban

useAtendimentoDetail

useAtendimentoExames

useCatalogSearch

useRealtimeAtendimentos

useValidatorName

services/

atendimentosService

atendimentoExamesService

catalogoExamesService

types/

atendimento

exame

catalogo

status

Recomendação sobre services

Mesmo usando supabase-js diretamente, centralizar operações em services.

Motivo:

- evita query espalhada em componente;
- facilita teste;
- facilita migração futura para API Routes.

8. Árvore de componentes

Kanban

KanbanPage

AppShell

KanbanFilters

RealtimeIndicator

KanbanBoard

KanbanColumn

AtendimentoCard

Card de validação

AtendimentoValidationPage

AppShell

AtendimentoHeader
StatusBadge
TotalSummary
ValidationActions
TermsNotFoundAlert
PatientForm
PrescriptionPanel
ConversationNotes
ExamList
CatalogAutocomplete
ExamRow
IncludedToggle
ExamTraceInfo
ExamEditor
OriginalQuoteViewer
ConfirmValidateDialog
ConfirmRejectDialog
ConflictDialog

9. Tempo real

9.1 Kanban

Subscription

Escutar mudanças em atendimentos.

Eventos relevantes:

- INSERT
- UPDATE
- DELETE, se houver

Uso

Quando o bot criar atendimento:

- Realtime recebe INSERT;
- card aparece na coluna aguardando_validacao.

Quando alguém abrir card:

- status muda para em_validacao;
- card muda de coluna.

Quando validar:

- status muda para validado;
- card muda de coluna.

Quando webhook/backend marcar enviado:

- status muda para enviado;
- card muda de coluna.

Recomendação técnica

Manter estado local normalizado por id.

Ao receber evento:

- se status pertence às colunas do Kanban, inserir/atualizar card;
- se saiu do filtro atual, remover;
- recalcular contadores com base no estado local.

9.2 Card de validação

Subscriptions recomendadas

Escutar:

1. atendimentos filtrado por id.
2. atendimento_exames filtrado por atendimento_id.

Uso

- Atualizar total_validado quando trigger recalcular.
- Refletir mudança de status.
- Refletir alteração feita por outro usuário.

Trade-off

Realtime em tela de detalhe pode gerar conflitos visuais se outro usuário editar ao mesmo tempo.

Recomendação:

- usar Realtime para detectar conflitos;
- se a tela local tiver alterações não salvas, mostrar aviso antes de sobrescrever;

- se não houver alterações locais, atualizar automaticamente.
-

10. Concorrência

10.1 Problema

A regra de negócio exige que, ao abrir um card, a web faça:

`status = 'em_validacao'`

Isso protege contra o bot, mas não identifica qual validador abriu o card, pois há login compartilhado.

10.2 Estratégia MVP — lock otimista por status

Ao abrir um atendimento:

1. tentar atualizar status para `em_validacao` somente se status atual for `aguardando_validacao`;
2. se o update retornar sucesso, liberar edição;
3. se não retornar sucesso, buscar atendimento;
4. se já estiver `em_validacao`, mostrar alerta de possível concorrência;
5. se estiver validado, enviado ou rejeitado, abrir como leitura ou bloquear edição.

Comportamento para segundo validador

Se outro validador abrir card já em `em_validacao`:

- mostrar banner: “Este atendimento já está em validação.”
- permitir leitura;
- permitir edição somente após confirmação “Assumir validação mesmo assim”.

Trade-off:

- Simples e não exige banco novo.
- Não impede totalmente duas pessoas de editarem ao mesmo tempo.
- Sem login individual, não há como saber com certeza quem está editando.

10.3 Recomendação futura — lock real

Solicitar mudança de banco se a operação exigir controle forte:

Novas colunas sugeridas em atendimentos:

- `em_validacao_por text`
- `em_validacao_em timestampz`
- `lock_expires_at timestampz`

Ou tabela separada:

- atendimento_locks

Com isso seria possível:

- mostrar quem abriu;
- expirar lock abandonado;
- impedir edição concorrente com mais segurança.

10.4 Conflito em edição de exames

Regra recomendada:

- cada alteração de exame é salva individualmente;
- se Realtime indicar que o mesmo exame foi alterado enquanto o usuário edita, mostrar alerta:
 - “Este exame foi atualizado em outra sessão.”
 - opções: “recarregar” ou “manter minha edição”.

Para o MVP:

- usar last-write-wins controlado por aviso visual;
- evitar telas com edição offline prolongada.

11. Segurança

11.1 Login compartilhado

A equipe usará uma única conta Supabase Auth.

Vantagens

- Simples.
- Rápido de implantar.
- Compatível com RLS já liberado para authenticated.

Desvantagens

- Não há rastreamento real por pessoa no Auth.
- Qualquer pessoa com a senha tem o mesmo nível de acesso.
- Auditoria fica limitada.

Recomendação

Manter login compartilhado no MVP, mas adicionar no front:

- campo “Validador atual”;
- salvar esse nome em validado_por ao validar;

- persistir localmente no navegador para não precisar digitar sempre.

Exemplo de valores:

- Ana
- João
- Maria
- Recepção
- Faturamento

Não é controle de segurança, é identificação operacional.

11.2 RLS

Usar RLS existente.

Permissões informadas:

- role authenticated tem SELECT/UPDATE em atendimentos;
- role authenticated tem FOR ALL em atendimento_exames.

Atenção

Como catalogo_exames é somente leitura, confirmar se role authenticated possui SELECT nessa tabela.

Dependência:

- se não houver SELECT em catalogo_exames, solicitar ajuste de policy/permissão.
-

11.3 Chaves e variáveis de ambiente

Variáveis públicas permitidas no front:

- NEXT_PUBLIC_SUPABASE_URL
- NEXT_PUBLIC_SUPABASE_ANON_KEY

Nunca expor:

- SUPABASE_SERVICE_ROLE_KEY
- tokens internos;
- secrets do webhook;
- credenciais do n8n;
- credenciais do WhatsApp.

Recomendação

Para o MVP, o client usa apenas:

- URL pública do Supabase;
- anon key;
- sessão authenticated via Supabase Auth.

O service_role não deve aparecer no bundle, no .env.local público ou em qualquer arquivo client-side.

12. Validações de formulário

12.1 Paciente

Alertas, não bloqueios duros

- paciente_nome vazio.
- paciente_cpf em formato estranho.
- paciente_nascimento ausente.
- paciente_cidade ausente.

Como o bot pode não extrair tudo corretamente, a tela deve permitir salvar, mas alertar.

12.2 Exames

Bloquear valores inválidos:

- preco negativo.
- prazo_dias negativo.
- jejum_horas negativo.

Permitir:

- preco = 0, mas alertar;
- preparo vazio, mas alertar;
- todos os exames desmarcados, mas exigir confirmação antes de validar.

12.3 Validar

Antes de validar, checar:

- telefone preenchido;
- status = em_validacao;
- total_validado carregado do banco;
- se há exames incluídos;
- se há termos não encontrados;

- se há exames ambíguos.

Recomendação:

- telefone vazio: bloqueio;
 - status diferente de em_validacao: bloqueio;
 - total zero: confirmação forte;
 - termos não encontrados: confirmação forte;
 - exames ambíguos: confirmação forte.
-

13. Casos de borda

13.1 Atendimento sem exames

Comportamento:

- mostrar estado vazio em exames;
- destacar termos não encontrados, se houver;
- permitir adicionar exame pelo catálogo;
- bloquear validação ou exigir confirmação forte.

Recomendação:

- bloquear validação sem exame incluído, salvo se houver regra de negócio clara para orçamento zerado.
-

13.2 Todos os exames desmarcados

Resultado:

- trigger deve recalcular total_validado como zero ou nulo, conforme implementação.

Comportamento da web:

- exibir total como R\$ 0,00 se vier zero;
 - se vier nulo, exibir “Total ainda não calculado”;
 - impedir validação normal;
 - permitir rejeição.
-

13.3 total_validado zerado

Comportamento:

- alerta vermelho;

- não calcular no front;
- botão Validar deve exigir confirmação forte ou ficar bloqueado.

Recomendação:

- bloquear validação por padrão.
-

13.4 Termo não encontrado virando exame

Fluxo:

1. usuário clica no termo não encontrado;
 2. autocomplete abre com o termo como busca inicial;
 3. usuário seleciona exame do catalogo_exames;
 4. sistema insere em atendimento_exames;
 5. trigger recalcula total_validado;
 6. termo original continua em termos_ao_encontrados como auditoria.
-

13.5 Validar sem telefone

Comportamento:

- bloquear.

Motivo:

- webhook depende de canal WhatsApp;
- validar sem telefone pode disparar erro externo.

Mensagem:

Não é possível validar: o atendimento não possui telefone do responsável.

13.6 Reabrir card já validado

Comportamento:

- abrir em somente leitura;
- mostrar aviso:
“Este atendimento já foi validado e pode já ter sido enviado ao cliente.”

Não permitir editar no MVP.

13.7 Card enviado

Comportamento:

- somente leitura;
 - exibir enviado_em, se disponível;
 - não permitir validar novamente.
-

13.8 Card rejeitado

Comportamento:

- somente leitura ou reabertura manual futura.

Como não há status de reabertura, não criar ação “reabrir” no MVP.

13.9 Falha ao validar

Possíveis causas:

- erro de rede;
- sessão expirada;
- RLS bloqueando update;
- status já alterado;
- webhook falha depois do update.

Comportamento:

- se update falhou: mostrar erro e manter card em edição;
 - se update funcionou: mostrar “validado”; o webhook é responsabilidade do backend;
 - acompanhar posterior status enviado via Realtime.
-

14. Plano de entrega em fases

Fase 1 — Base do projeto e autenticação

Entregas

- Projeto Next.js configurado.
- Tailwind configurado.
- Supabase client configurado.
- Tela de login.
- Controle de sessão.
- Layout base da aplicação.

Critérios de aceite

- usuário consegue logar com conta compartilhada;
 - usuário não autenticado não acessa Kanban;
 - sessão persiste ao recarregar página;
 - nenhuma service_role key aparece no front.
-

Fase 2 — Kanban inicial

Entregas

- Tela Kanban.
- Colunas por status:
 - aguardando_validacao
 - em_validacao
 - validado
 - enviado
 - rejeitado
- Contadores por coluna.
- Cards resumidos.
- Navegação para card detalhado.

Critérios de aceite

- cards carregam do Supabase;
 - contadores batem com os cards exibidos;
 - clique abre atendimento correto;
 - estado vazio funciona;
 - erro de carregamento é tratado.
-

Fase 3 — Realtime no Kanban

Entregas

- Subscription em atendimentos.
- Atualização automática de cards.
- Mudança de coluna quando status muda.
- Indicador de conexão.

Critérios de aceite

- novo atendimento criado pelo bot aparece sem refresh;
 - card muda para em_validacao quando aberto;
 - card muda para validado ao validar;
 - card muda para enviado quando backend atualizar.
-

Fase 4 — Tela de validação

Entregas

- Cabeçalho completo.
- Bloco do paciente editável.
- Bloco da prescrição.
- Observações da conversa.
- Lista de exames.
- Exibição de termos não encontrados.
- Exibição de cotacao_original.

Critérios de aceite

- dados do atendimento carregam corretamente;
 - exames carregam por atendimento_id;
 - campos somente leitura não são editáveis;
 - termos não encontrados aparecem em destaque;
 - total validado aparece a partir do banco.
-

Fase 5 — Edição de exames e paciente

Entregas

- Editar dados do paciente.
- Editar preço, prazo, jejum e preparo.
- Marcar/desmarcar incluído.
- Definir editado_manual = true em alterações.
- Atualizar total após trigger.

Critérios de aceite

- alteração em exame persiste no banco;

- total_validado muda após trigger;
 - web não grava total_validado;
 - exame desmarcado deixa de compor total;
 - edição inválida é bloqueada ou alertada.
-

Fase 6 — Autocomplete do catálogo

Entregas

- Busca em catalogo_exames.
- Seleção de exame.
- Inserção em atendimento_exames.
- Preenchimento automático de:
 - sku
 - nome
 - preco
 - preparo
 - prazo_dias
 - jejum_horas

Critérios de aceite

- usuário busca exame pelo nome;
 - usuário adiciona exame ao atendimento;
 - exame entra com incluído = true;
 - exame entra com editado_manual = true;
 - total recalcula após inserção.
-

Fase 7 — Validação, rejeição e concorrência

Entregas

- Ao abrir card, update para em_validacao.
- Confirmação de validação.
- Confirmação de rejeição.
- Campo/seletor validado_por.
- Tratamento de card já em validação.

Critérios de aceite

- abrir card muda status de aguardando_validacao para em_validacao;
 - validar muda status para validado;
 - rejeitar muda status para rejeitado;
 - validação pede confirmação;
 - validação bloqueia duplo clique;
 - card já validado abre em somente leitura.
-

Fase 8 — Testes, polimento e entrega

Entregas

- Checklist funcional completo.
- Tratamento de erros.
- Ajustes responsivos.
- Melhorias de UX.
- Documentação curta de uso.

Critérios de aceite

- fluxo ponta a ponta aprovado;
 - Realtime funcionando;
 - sem cálculo indevido de total no front;
 - sem exposição de chaves sensíveis;
 - equipe consegue operar sem suporte técnico.
-

15. Riscos e dependências

15.1 Login compartilhado

Risco:

- baixa rastreabilidade real.

Mitigação:

- campo validado_por;
 - futuramente, login individual por funcionário.
-

15.2 Concorrência limitada

Risco:

- dois validadores podem editar o mesmo card.

Mitigação MVP:

- lock por status = em_validacao;
- alerta de conflito;
- Realtime no detalhe.

Dependência futura:

- criar colunas de lock real.
-

15.3 Falta de motivo de rejeição

Risco:

- rejeições sem explicação.

Mitigação MVP:

- rejeição simples.

Dependência futura:

- adicionar motivo_rejeicao text;
 - ou tabela de histórico.
-

15.4 Busca limitada no catálogo

Risco:

- autocomplete ruim se catálogo for grande ou sinônimos estiverem mal estruturados.

Mitigação MVP:

- busca simples por nome, sku, sinonimos.

Dependência futura:

- índice full-text;
 - função RPC;
 - ranking por similaridade.
-

15.5 RLS em catalogo_exames

Risco:

- autocomplete não funciona se authenticated não tiver SELECT.

Dependência:

- confirmar ou criar policy SELECT para catalogo_exames.
-

15.6 Total recalculado de forma assíncrona

Risco:

- usuário edita exame e o total demora a refletir.

Mitigação:

- mostrar estado “atualizando total” após edição;
 - reler atendimento;
 - usar Realtime para capturar mudança.
-

15.7 Validar dispara efeito externo

Risco:

- envio indevido ao cliente.

Mitigação:

- modal de confirmação;
 - bloqueio de duplo clique;
 - botão com loading;
 - restrição para status em _validacao.
-

16. Dependências de banco a solicitar somente se necessário

Não assumir agora, mas listar para evolução:

1. atendimentos.em_validacao_por text
2. atendimentos.em_validacao_em timestampz
3. atendimentos.lock_expires_at timestampz
4. atendimentos.motivo_rejeicao text
5. atendimentos.rejeitado_por text
6. atendimentos.rejeitado_em timestampz
7. tabela atendimento_eventos para auditoria
8. índice full-text em catalogo_exames
9. RPC de busca de catálogo com ranking

10. policy SELECT em catalogo_exames, se ainda não existir

17. Checklist de testes

17.1 Autenticação

- Usuário consegue acessar tela de login.
 - Login compartilhado autentica corretamente.
 - Usuário não logado é redirecionado.
 - Logout encerra sessão.
 - Recarregar página mantém sessão.
 - Nenhuma chave sensível aparece no front.
-

17.2 Kanban

- Carrega cards de aguardando_validacao.
 - Carrega cards de em_validacao.
 - Carrega cards de validado.
 - Carrega cards de enviado.
 - Carrega cards de rejeitado.
 - Contadores batem com a quantidade de cards.
 - Busca por protocolo funciona.
 - Busca por paciente funciona.
 - Busca por telefone funciona.
 - Busca por médico funciona.
 - Estado vazio aparece corretamente.
 - Erro de Supabase mostra mensagem clara.
-

17.3 Realtime

- Novo atendimento criado pelo bot aparece no Kanban sem refresh.
- Mudança de status move card de coluna.
- Card validado aparece na coluna validado.
- Card enviado aparece na coluna enviado.
- Indicador de conexão funciona.

- Reconnect não duplica cards.
-

17.4 Abertura do card

- Card aguardando_validacao muda para em_validacao ao abrir.
 - Card já em_validacao mostra alerta.
 - Card validado abre em somente leitura.
 - Card enviado abre em somente leitura.
 - Card rejeitado abre em somente leitura ou modo restrito.
 - Falha no update de status mostra erro.
-

17.5 Dados do paciente

- Nome carrega corretamente.
 - CPF carrega corretamente.
 - Nascimento carrega como texto.
 - Cidade carrega corretamente.
 - Convênio carrega corretamente.
 - Unidade carrega corretamente.
 - Alterações persistem.
 - Campos vazios geram alerta, não erro fatal.
-

17.6 Prescrição

- Médico solicitante aparece.
 - Qualidade da imagem aparece.
 - Observações da prescrição aparecem.
 - Campo somente leitura não permite edição.
 - Campo editável persiste, se habilitado.
-

17.7 Exames

- Lista carrega todos os exames do atendimento.
- Ordenação por ordem funciona.
- Editar preço persiste.

- Editar prazo persiste.
 - Editar jejum persiste.
 - Editar preparo persiste.
 - Alterar exame marca editado_manual = true.
 - Desmarcar incluído remove do total.
 - Remover do orçamento usa incluído = false.
 - Exame ambíguo aparece destacado.
 - match_por, score e termo_original aparecem como rastreio.
 - Campos de rastreio não são editáveis.
-

17.8 Catálogo

- Autocomplete busca em catalogo_exames.
 - Busca por nome funciona.
 - Busca por SKU funciona.
 - Busca por sinônimo funciona, se disponível.
 - Selecionar exame copia os dados corretos.
 - Inserção cria linha em atendimento_exames.
 - Novo exame entra com incluído = true.
 - Novo exame entra com editado_manual = true.
 - Total atualiza após inserção.
-

17.9 Total

- total_validado aparece a partir de atendimentos.
 - Front não grava total_validado.
 - Alterar preço de exame altera total após trigger.
 - Desmarcar exame altera total após trigger.
 - Adicionar exame altera total após trigger.
 - Total zero gera alerta.
 - Total nulo é tratado visualmente.
-

17.10 Validação

- Botão Validar abre modal de confirmação.
 - Modal mostra telefone.
 - Modal mostra total validado.
 - Modal exige ou recomenda validado_por.
 - Confirmar muda status para validado.
 - validado_em é preenchido.
 - validado_por é preenchido.
 - Botão fica em loading.
 - Duplo clique não dispara duas ações.
 - Validar sem telefone é bloqueado.
 - Validar atendimento já validado é bloqueado.
 - Após validar, tela entra em somente leitura.
-

17.11 Rejeição

- Botão Rejeitar abre confirmação.
 - Confirmar muda status para rejeitado.
 - Card vai para coluna rejeitado.
 - Atendimento rejeitado não pode ser validado sem regra de reabertura.
-

17.12 Teste end-to-end principal

Fluxo completo:

1. Bot cria atendimento com status = aguardando_validacao.
2. Kanban mostra novo card em aguardando_validacao.
3. Validador abre card.
4. Web muda status para em_validacao.
5. Validador corrige paciente.
6. Validador edita exame.
7. Validador desmarca exame incorreto.
8. Validador adiciona novo exame via catalogo_exames.
9. Banco recalcula total_validado.
10. Web exibe novo total_validado.

11. Validador confirma “Validar”.
 12. Web atualiza status = validado.
 13. Webhook envia orçamento ao cliente.
 14. Backend marca ou confirma envio.
 15. Kanban mostra card em enviado, quando o backend atualizar.
 16. Conferir no WhatsApp que o cliente recebeu o orçamento correto.
-

18. Recomendação final de execução

A entrega deve priorizar primeiro o fluxo operacional mínimo:

Login → Kanban → Abrir card → Status em_validacao → Editar exames → Ver total do banco → Validar

Depois entram melhorias:

- filtros;
- Realtime refinado;
- UX de conflito;
- auditoria;
- busca avançada no catálogo.

A decisão mais importante é preservar a separação de responsabilidades:

- **Bot cria o atendimento.**
- **Web valida e edita.**
- **Banco calcula o total.**
- **Webhook envia ao cliente.**

Essa divisão reduz risco de duplicidade de regra, evita divergência de valores e mantém o Supabase como fonte de verdade.